

DRIVE247 · PRODUCT SPECIFICATION

Fleet Health

Vehicle maintenance, inspections and cost intelligence —
a complete end-to-end flow, grounded in the live system.

Origin Brandon Rockow (Bonzah) — maintenance as a retention/USP feature

Status Draft for review — no code written, no schema applied

Date 15 August 2026

Basis Live production database + full monorepo audit (12 research tracks, 3 adversarial reviews)

Scope apps/portal · apps/booking · supabase/functions · public schema

0 · How to read this document

This is not a wish-list. Every claim below was checked against the **live production database** and the actual repository. Where the popular assumption is wrong, it says so. Where a number is inflated by demo data, the real number is given.

The single most important finding is this: **Drive247 already contains roughly 60% of a maintenance system**. It is scattered, partly orphaned, partly broken, and entirely un-surfaced — but it exists. The work is far less about building new machinery than about connecting, repairing and exposing what is already there. Building a fresh "maintenance module" alongside it would create a second, competing source of truth for cost and availability, and would actively make the product worse.

- | | | | |
|---|------------------------------------|----|-------------------------------|
| 1 | Verdict & strategy | 7 | Data model |
| 2 | Brandon's proposal, point by point | 8 | The end-to-end flows (F1–F13) |
| 3 | What already exists | 9 | Edge-case register |
| 4 | Day-one data reality | 10 | Non-functional requirements |
| 5 | Defects that must be fixed first | 11 | Phasing & sequencing |
| 6 | Core architecture | 12 | Decisions required from you |

1 · Verdict & strategy

Brandon is right that maintenance is the correct next feature, and right about why: it is the only thing that gives an operator a reason to open Drive247 on a day with no bookings. But the version described in his message — health scores, work orders, parts/labour, predictive maintenance, telematics — is a Fleetio clone that would take months and would land on a data substrate that cannot support it.

THE BLOCKING CONSTRAINT

Mileage-interval maintenance ("oil change every 5,000 miles") is the centrepiece of the proposal. Across the entire production database, **exactly 5 vehicles** have both a current odometer reading and a last-service mileage. Of 196 real vehicles, 40 have any odometer reading at all and 10 have a last-service date. There is **no UI anywhere in the product** that lets a human type a starting odometer — the only writer is the key-handover screen. A mileage-based feature therefore has almost nothing to compute against on day one, and no way for an operator to fix that.

The strategy that follows from that

Invert Brandon's ordering. He proposed schedules first and compliance later. The data says the opposite:

1. **Lead with legal compliance, not servicing.** Registration/inspection dates are the best-populated maintenance data in the system and the highest-liability gap. Renting out a vehicle whose inspection has lapsed is a strict-liability offence for the operator and typically voids their own motor policy. Today **nothing** in the product reads those dates for availability — a vehicle with an expired inspection is fully bookable and publicly listed. That is both the strongest adoption hook ("this stops you renting an illegal car") and the most defensible reason to build.
2. **Make the odometer capturable before making it computable.** Every mileage rule is blocked on a capture path that does not exist. Fix capture first; intervals become possible as a consequence.
3. **Make the maintenance hold real.** The promise "automatically take unsafe vehicles out of availability" is currently unenforceable — see §5.1. Until a hold cannot be bypassed, every downstream feature is decorative.
4. **Design for one person, not a team.** 17 of the 23 vehicle-owning tenants have exactly **one** staff login. There are 4 manager users and 2 ops users on the entire platform against 47 head_admins. Every "assign it to ops / the manager reviews it" workflow is fiction.

POSITIONING

Brandon's instinct to avoid calling it "vehicle maintenance management" is correct, and there is a technical reason to insist on it: **the word "maintenance" is already taken three times** in this codebase (§5.9). **Fleet Health** is the recommended name for the product surface, and every new database object should be namespaced `vehicle_*` to avoid colliding with the existing `tenant-health` and `platform-maintenance` systems.

2 · Brandon's proposal, point by point

Each item from the message, assessed against the live system.

Proposal	Assessment	What it actually costs	Verdict
1. Vehicle Health score /100	A single number is the wrong primitive here. With 20% odometer coverage a score would be mostly imputed from absent data, and a confident "92/100" on a vehicle nobody has looked at is worse than silence. It also implies a safety judgement the product must not make (§10.4).	Cheap to render, expensive in liability and trust.	RESHAPE — ship a status (OK / Attention / Overdue / Unknown) with the reasons listed, not a score. "Unknown" must be first-class.
1b. "What needs my attention" in 5 seconds	Correct and the real product. This is the daily-open hook.	Low — one list view.	BUILD FIRST
2. Mileage/time maintenance schedules	Right model, wrong sequencing. Time-based works today; mileage-based is computable for 5 vehicles platform-wide.	Schedule engine is moderate. The odometer capture it depends on is the real cost.	SPLIT — time-based in P1, mileage-based in P2 behind capture.
2b. "Service expected in ~12 days based on utilisation"	The best idea in the message, and genuinely differentiating. Also directly implementable: the existing <code>reminders</code> table is date-based, so a forecast that converts miles → date drops straight into working infrastructure.	Low once mileage exists. Median fleet burn is 166 mi/day (p90 320) from the 25 rentals with both readings.	BUILD — but label it an estimate and show the confidence.
3. Maintenance ↔ booking conflict detection	The strongest differentiator versus Fleetio, and Drive247 genuinely has the data. But it must run in the <i>other</i> direction too, which the message misses: scheduling maintenance over an already-paid booking is the dangerous case (§8.6).	Moderate. Requires the availability fix in §5.1 to mean anything.	BUILD — bidirectional.
4. Pre/post-rental inspections	Half-built and never used. Photo capture, AI damage diff (<code>detect-vehicle-damage</code> , GPT-4o vision) and a review UI all exist. <code>rental_damage_reports</code> has 0 rows — it has never run once in production. Structural problem: 9 tenants use lockbox handover where nobody is present to inspect.	Low to activate, high to make trustworthy (§5.8 evidence chain).	ACTIVATE + HARDEN
5. Maintenance issues (priority/category/status)	Sound, and the natural home for "something is wrong with this car" independent of a schedule.	Low — one table, one list.	BUILD
6. Work orders (parts/labour/garage/invoice)	Correctly identified as V2. Two hard blockers: the garage is a free-text string with no vendor entity, and the accounting sync is receivables-only (Xero hardcodes <code>ACCREC</code>) so supplier bills cannot flow.	High. Parts/labour split also drags in tax treatment the ledger cannot represent (§5.6).	DEFER TO P3 — ship "job + cost + receipt photo" instead.

Proposal	Assessment	What it actually costs	Verdict
7. Maintenance calendar beside bookings	Already 80% built. The rentals calendar already renders <code>blocked_dates</code> alongside rentals and already has realtime wired.	Low — extend an existing view.	EXTEND
8. Maintenance dashboard	Good, but must not become a 15th nav item nobody opens. The signal has to appear where vehicles are <i>chosen</i> , not on a tab.	Low.	BUILD — plus inline chips (\$8.11).
9. Maintenance cost per vehicle	The infrastructure exists (<code>pnl_entries</code> , <code>view_pl_by_vehicle</code> , <code>get-vehicle-profitability</code>) and service costs already flow into it automatically. But the ledger books collected sales tax and security deposits as <i>Revenue</i> , has no currency column, and expenses the full purchase price with no depreciation. Cost figures are sound; profitability ratios are not .	Low for cost. Not safely deliverable for ratios.	COST YES, RATIOS NO — see §5.6.
10. Trax AI maintenance queries	Very achievable and cheaper than expected. Trax answers aggregate questions from an authoritative live SQL blob (<code>get_rag_metrics</code>), not from the vector store — so adding maintenance means extending one SQL function. Caveat: the vector corpus is frozen (80,565 unprocessed sync rows, no cron) and must not be relied on.	Low via the metrics channel.	BUILD — metrics channel only.
11. Predictive maintenance / telematics	Correct to defer. Worth knowing: the Tesla integration is the only telematics in the product, it never reads odometer, and it is dead in production — every call 404s and all 6 rows are mock seeds. <code>has_tracker</code> (291 vehicles) is a checkbox no logic reads.	High, and currently blocked on a broken integration.	P3+
12. Renter damage → claim connection	The most commercially interesting idea and the most legally dangerous. There is no claim artefact anywhere in the product. More seriously, the evidence chain will not survive a dispute: photos sit in a public bucket the renter can write to and delete, and no handover records who performed it (<code>handed_by</code> NULL on all 160).	Evidence hardening is a prerequisite, not a nice-to-have.	HARDEN FIRST then P3

3 · What already exists

Reusable today. Building parallel versions of any of these is the main risk to the project.

3.1 The service + cost spine **WORKS WELL**

`service_records` is already the canonical completed-work log, and it is wired far better than anyone expects. A single trigger does three jobs:

- `update_vehicle_last_service()` re-derives `vehicles.last_service_date` and `last_service_mileage` from the newest record.
- `upsert_service_pnl_entry()` writes a `pnl_entries` Cost/Service row keyed `reference='service:<id>'` — an idempotent upsert, correctly deleted on removal.

- Deletion cleans both up.

ARCHITECTURAL CONSEQUENCE

A completed maintenance job must settle into `service_records`. Do that, and P&L posting, the service baseline and the vehicle's last-service fields all update for free, idempotently. This is the single cleanest decision in the whole design — and the reason a separate `maintenance_costs` table must never be created.

3.2 The availability + hold mechanism EXISTS BUT BROKEN

A `Maintenance` vehicle status and an auto-release reconciler already exist and are live:

- `vehicles.status = 'Maintenance'` + a `blocked_dates` row covering today = "off the road".
- Cron job 56 (`0 1 * * *`) runs `sync_vehicle_maintenance_status()`, returning the vehicle to Available/Rented once no block covers today.
- `VehicleStatusBadge` already renders Maintenance with a wrench and amber styling — zero UI work needed.

Caveat: no vehicle has ever held this status, the only writer is `swap_rental_vehicle`, there is no UI to set it, and the reconciler has several defects (§5.1–5.3).

3.3 The reminder engine WORKS, BUT NOT THE WAY IT LOOKS

`reminders` is a generic polymorphic table (`rule_code`, `object_type`, `object_id`, `due_on`, `remind_on`, `severity`, `status`, `snooze_until`, `context`) and vehicle-scoped reminders **already exist and fire**: `MOT_30D`, `MOT_7D`, `MOT_0D`, `TAX_30D`, `TAX_7D`.

DO NOT USE THE RULE ENGINE

The "designed" pipeline (`reminder_rules` → `reminders-generate` → `process-reminders`) is **dead**: `reminder_rules` has **0 rows**, so the generator early-returns before reaching its vehicle loop; none of those six edge functions has a cron job; and the one reminder cron that does exist posts to a **foreign Supabase project**. Every alert that actually works in production is hand-rolled — code that learns a fact writes a `reminders` row directly with its own dedupe guard. Follow the working pattern, not the designed one.

3.4 Components already written but never mounted

Four ready-made pieces have **zero importers** — Phase 1 is partly a matter of rendering them:

Component	What it already does
<code>vehicle-compliance-panel.tsx</code>	"Inspection & Registration Status" card — renders MOT/tax due status, pulls critical/warning counts from <code>useRemindersByObject('Vehicle', id)</code> , deep-links to the reminders page. Already wired to the reminder system.
<code>service-history-table.tsx</code>	Service table with working Edit and Delete, currency formatting, and a P&L badge. Wired to exactly the mutation signatures <code>useVehicleServices</code> exposes.
<code>last-service-card.tsx</code>	Last-service summary card.
<code>use-vehicle-events.ts</code>	Timeline reader for <code>vehicle_events</code> . Currently returns zero rows for every tenant — all 84 rows have <code>tenant_id</code> NULL while the hook filters on it. Needs a writer fix plus a backfill before it works.

3.5 Everything else worth reusing

Asset	Role in Fleet Health	Condition
<code>vehicle_expenses</code>	Cost lines with vendor, receipt URL, categories (Service, Repair, Tyres, Valet already in live use)	Healthy — but see double-count risk §5.6
<code>blocked_dates</code>	The off-road primitive	Date-only; 59 overlapping pairs live; RLS off; in the realtime publication

Asset	Role in Fleet Health	Condition
<code>rental_key_handovers.mileage</code>	The only odometer source in the product	Optional, no guards, 68% populated
<code>rental_handover_photos + detect-vehicle-damage</code>	Condition evidence + AI before/after diff	Built, never run (0 reports)
<code>pnl_entries / view_pl_by_vehicle / get-vehicle-profitability</code>	Cost reporting	Works for cost; unsafe for ratios (\$5.6)
<code>get_rag_metrics</code> RPC (Trax)	The authoritative aggregate channel Trax answers from	Healthy — extend this, not the vector store
<code>swap_rental_vehicle</code> RPC	The transactional template for a maintenance hold; also the remedy for taking a rented car off the road	Healthy — copy its shape
Rentals calendar + <code>use-calendar-blocks</code>	Maintenance-beside-bookings view	Healthy; delete path missing a tenant predicate

4 · Day-one data reality

These numbers decide what can ship. **Demo data must be excluded** — the `test` tenant alone holds 242 vehicles and inflates every naïve count by roughly 6×.

196	40	10	5	23	17
REAL VEHICLES	WITH ODOMETER	WITH LAST SERVICE DATE	MILEAGE-INTERVAL CAPABLE	TENANTS WITH VEHICLES	HAVE 1 STAFF LOGIN

Signal	Real	Inc. demo	Consequence
Vehicles with <code>current_mileage</code>	40 / 196	248	Mileage rules dormant for 80% of the fleet
Vehicles with both mileage anchors	5	5	Mileage-interval logic is computable for 5 cars platform-wide
Registration / inspection dates	~38 / ~41	268 / 271	Best-populated signal, but still only ~20% of real vehicles
<code>service_records</code>	41 rows, 8 tenants	—	27 belong to one tenant; 21 have no mileage; 22 typed "Other"/NULL
Handovers with a reading	109 / 160	—	Return readings are worst: 26 of 53
AI damage reports ever generated	0	0	Feature built, never used — an adoption warning, not a capability
Unread notifications	14,325 / 14,448	—	99.1% unread. Adding a stream here guarantees it is never seen
Reminders ever snoozed	0 of 598	—	Snooze cannot be the alert-fatigue control; nobody uses it
Fleets with zero odometer coverage	nealcorentals 0/22, amgroadside 0/15, goniko 0/15, sdrentals 0/12		The largest real fleets are at absolute zero

THE FIRST-IMPRESSION PROBLEM

An operator with 22 vehicles who opens Fleet Health on day one sees 22 rows of "Unknown". That reads as broken software, not as missing input — and they cannot fix it, because **no screen in the product lets a human enter an odometer reading**. A bulk setup flow and a manual odometer control are therefore **gating requirements**, not enhancements. When coverage is below threshold, the module must render a setup screen, never an all-unknown table.

5 · Defects that must be fixed first

Each of these breaks a promise Fleet Health would make. They are ordered by severity.

5.1 A maintenance hold is bypassable **BLOCKER**

The DB trigger `check_rental_overlap()` is the only server-side booking guard, and it compares rentals to rentals only. It **never reads `blocked_dates`** and never reads `vehicles.status`. Every block check in the product is a client-side `SELECT` before an `INSERT`, duplicated across at least **8 call sites** in booking and portal, with no transaction and no lock. Zero of the ~300 edge functions reference `blocked_dates`.

Worse, the portal's own manual booking form **deliberately ignores vehicle-specific blocks** (`rentals/new/page.tsx:482` — "Admin portal only enforces global blocks"), which covers 199 of the 203 live block rows. That is the widest bypass and it is on the screen operators use most.

REQUIRED

Add a second `EXISTS` clause to `check_rental_overlap` against `blocked_dates`, raising a **distinct error code** so the UI can say "in service" rather than "double-booked". It must respect the function's existing `check_start` fast-path variable — the live function sets `check_start := OLD.end_date` on a pure extension, and ignoring that would make every auto-extend fail on any vehicle with a historical block. Also specify what `auto-extend-rentals` does when it now hits this exception mid money-flow: it must pause with a reason, not die inside a ledger write.

5.2 Every hold releases hours early **BLOCKER**

The database runs in UTC and cron 56 fires at `0 1 * * *` UTC. Every tenant timezone is **behind** UTC (America/New_York 31, America/Chicago 8, America/Los_Angeles 4, America/Denver 3). When the reconciler evaluates `end_date >= CURRENT_DATE`, the UTC date has already rolled while it is still 18:00–21:00 the previous day locally.

So on the final day of **every** maintenance window, for **every** tenant, the vehicle flips to Available 3–7 hours early — while the client-side block checks still consider the day blocked. Status and `blocked_dates` disagree, and during that window the portal booking form will rent the car with no friction at all.

5.3 The reconciler is unsafe as written **BLOCKER**

`sync_vehicle_maintenance_status()` is an unconditional bulk `UPDATE` with **no tenant filter**, no `is_disposed` guard, and no awareness of whether work is still open. It competes with three TypeScript rental-close paths, at least four edge functions, and three SQL functions (`dispose_vehicle`, `undo_vehicle_disposal`, `swap_rental_vehicle`) — none of which read the current value before writing, seven of which set `Available` unconditionally. There is no status history and no audit of the prior value, so a wrong state is undiagnosable.

Related: `blocked_dates` has **59 overlapping pairs live** and no unique or exclusion constraint. "Is this vehicle off the road" is a range-union question, not a row lookup — so completing one job and deleting its block may leave the car stuck in Maintenance behind a second block, with no UI explaining why.

5.4 Odometer capture is unguarded **BLOCKER**

`use-key-handover.ts:604` is the only writer of `vehicles.current_mileage` in the repo, and it:

- guards with `if (mileage)` — so a legitimate **0 on a brand-new vehicle is falsy** and silently skips both the vehicle write and the excess-mileage calculation;

- writes on **both** giving and receiving with no monotonic guard, so readings can go backwards (one vehicle is already below its own highest recorded reading);
- has **no tenant predicate** on the vehicles UPDATE;
- is optional — a "Continue Without Reading" dialog waives it, and 51 of 160 handovers have no reading.

Separately, `update_vehicle_last_service()` takes the newest record **regardless of `service_type`**, and NULLs `last_service_mileage` when the newest record has no mileage. So a quick "brakes, £340" entry with no odometer **permanently destroys that vehicle's interval baseline**. It must **COALESCE** to the newest non-NULL mileage.

5.5 Mid-rental swaps corrupt mileage attribution **ARMED TODAY**

`rental_key_handovers` has **no `vehicle_id` column** and is UNIQUE on `(rental_id, handover_type)` — so it structurally cannot record which physical car a reading belongs to, nor more than two readings. One live rental has been swapped **twice**, meaning three vehicles on one rental.

Four live rentals currently have a giving reading recorded *before* a swap and no return reading yet. The moment a return odometer is typed, the delta spans two physical vehicles and the reading is stamped onto whichever car happens to be attached. This is armed, not theoretical.

5.6 The ledger cannot support profitability claims **DECISION NEEDED**

Ledger property	Reality
Sales tax	Booked as Revenue — 3,443 rows / \$203,283. Per-vehicle "net profit" is overstated by all tax collected. All those rows have <code>reference = NULL</code> , so they cannot be deduped or reversed.
Security deposits	Also on the Revenue side (168 rows).
Currency	<code>pnl_entries</code> has no currency column . All tenants are USD today, but a UK → UAE migration is in flight; the day a tenant flips, all history silently reinterprets.
Tax on costs	No tax column anywhere in <code>pnl_entries</code> or <code>ledger_entries</code> . Every post-booking charge is untaxed by construction, and 16 tenants have sales tax enabled at 3–13%.
Capex vs opex	Acquisition is \$23.7M expensed in a single period against \$8.4k of service. No depreciation, no useful-life, no capital-improvement concept. A \$6,000 engine replacement books as a period expense.
Double-count risk	<code>service_records</code> posts <code>service:<id></code> ; <code>vehicle_expenses</code> posts <code>vexp:<id></code> . Both land in category Service with no FK and no reconciliation. A "log the job / attach the invoice" UI invites the operator to enter both — and because the bucket just looks larger, the error is invisible. Zero such pairs exist today , so this is a defect Fleet Health would <i>introduce</i> .

CONSEQUENCE

Brandon's \$9 — "maintenance / revenue = 21.8%, the C63 costs 2.2× the fleet average" — is **not safely deliverable** on this ledger. Maintenance **cost** per vehicle is sound and should ship. Any ratio against revenue, and any disposal recommendation, must wait until tax and deposits are reclassified out of Revenue.

5.7 Compliance is modelled in the wrong jurisdiction **DECISION NEEDED**

The only compliance fields are `mot_due_date` and `tax_due_date` — UK vocabulary. But all 46 tenants are USD, tax rates are US state sales-tax shaped, and tenant names are LLCs. There is **no country or state column on `tenants`** (only a free-text address), and `vehicles.garaging_state` is populated on 204 vehicles **all belonging to the demo tenant** and zero real ones.

MOT is one national annual test. US state inspection is not: New York is annual safety + emissions, Texas annual, California biennial smog with model-year exemptions, Florida has none — and registration renewal is a separate obligation on a separate cadence. A feature keyed on `mot_due_date` tells a Florida operator to book a test that does not exist.

5.8 The evidence chain will not survive a dispute BLOCKER FOR ANY CHARGE

- Handover photos live in a **public bucket** with policies granting every authenticated user SELECT, UPDATE and DELETE — **including the renter being charged**, who authenticates against the same Supabase project.
- `handed_by` is NULL on all 160 handovers; `uploaded_by` is NULL on all 1,119 photos. There is no named custodian for any evidence.
- `detect-vehicle-damage` upserts and **wipes** `reviewer_notes / reviewed_by` **on every re-run**, and its findings reference photos by positional index into a mutable list.
- Client-side upload rejects files over 10MB or non- `image/*` by **silently returning** — no toast, no error. That is exactly what a modern iPhone produces, and the staff member walks away believing the photos uploaded.

5.9 Three-way naming collision MUST NAMESPACE

Existing name	What it actually means
<code>maintenance_banner_*</code> , <code>MaintenanceBanner</code>	Platform software downtime notice ("some features temporarily unavailable"). Nothing to do with vehicles. Present in both booking and portal.
<code>maintenance_runs</code>	A data-operations batch job table (<code>payments_processed</code> , <code>revenue_recalculated</code>).
<code>health_score_*</code> , <code>evaluate-health-</code> <code>scores</code> , <code>tenant_health_*</code> , <code>rentals.health_severity</code>	Tenant churn risk . A live cron. Nothing to do with vehicles.

Every new object must be namespaced `vehicle_*`. Never introduce a bare `health_score` or `maintenance_*` table.

5.10 Isolation and realtime hazards

- RLS is **disabled on 72 public tables**, including every table Fleet Health touches: `vehicles`, `service_records`, `vehicle_events`, `blocked_dates`, `rental_key_handovers`, `reminders`. Isolation currently depends entirely on a hand-written `.eq('tenant_id', ...)` in each hook.
- `blocked_dates` is **in the realtime publication with RLS off**. A free-text reason like "engine warning light" or a garage name is broadcast to any holder of the public anon key — which ships in the booking bundle. Keep narrative text off `blocked_dates`; store a structured `reason_code` there and the detail on the tenant-scoped table.
- `use-calendar-blocks.ts` deletes blocks by id with **no tenant predicate** — a safety hold is cross-tenant deletable.
- `vehicles` is **not** in the realtime publication, so a status flip propagates to nobody. It must **not** be added while RLS is off — that would broadcast every tenant's fleet. Follow the documented `tenant_subscriptions` pattern instead: touch a published row, re-read over authenticated PostgREST.
- Route protection **fails open**: any route absent from `ROUTE_TO_TAB` is reachable by every manager. Two edge functions carry hardcoded `ALLOWED_TAB_KEYS` lists already 10 keys behind `permissions.ts`.
- `vehicle_events` has `tenant_id` NULL on all 84 rows, so its hook returns zero rows for everyone. Needs a writer fix plus backfill before the timeline works.

6 · Core architecture

6.1 Five principles

1. **Observation, not state**. Mileage is a stream of dated, attributed observations. `vehicles.current_mileage` becomes a derived cache (MAX over observations), never a last-writer-wins scalar.
2. **One settlement path**. Completed work settles into `service_records`. P&L, baseline and last-service fields follow automatically. No parallel cost table.
3. **One transition authority**. A single `SECURITY DEFINER` RPC is the sole writer of `vehicles.status`, modelled on `swap_rental_vehicle`, writing an audit row on every transition.
4. **Unknown is a first-class state**. Distinct from "not due", counted and surfaced. On day one it is the majority.

5. **Enforce at the database.** Any rule the product promises to enforce lives in `check_rental_overlap` , not in a client filter. Client checks are advisory UX only.

6.2 The evaluation model

Health status is **computed by cron, cached on a row, and recomputed on specific triggers**. Compute-on-read is too expensive for a fleet list; compute-on-write misses the pure clock events (a due date passing changes nothing in the database).

Recompute triggers — the complete set

- 1 A handover mileage write (either direction)
- 2 A manual odometer correction
- 3 `service_records` insert / update / delete
- 4 A maintenance schedule created, edited or deleted
- 5 A vehicle swap on an open rental
- 6 A compliance date edited on the vehicle
- 7 The nightly sweep — the backstop, and the **only** thing that can catch a due date simply passing

6.3 Status derivation

Status	Condition	Behaviour
Unknown	No anchor exists for any configured rule (no odometer, or no baseline)	Counted separately. Drives the setup prompt. Never rendered as "OK".
OK	All rules outside their lead window	Silent
Attention	Any rule inside its lead window, or projected to breach before the next booking ends	Amber chip wherever the vehicle appears
Overdue	Any rule past due	Red chip; warn at point of rental
Not road legal	A compliance obligation has lapsed	Blocks or hard-warns at point of rental — see §12 D1
Off road	An open hold covers today	Excluded from availability at the database level

6.4 Forecasting

Brandon's "service expected in ~12 days" converts a mileage gap into a date, so it can flow into the existing date-based reminder table:

```
projected_date = today + (miles_remaining ÷ daily_burn)
```

`daily_burn` resolves in priority order: (1) this vehicle's own observed burn between its last two readings; (2) the tenant's fleet median; (3) the platform median (**166 mi/day**, p90 320, from the 25 rentals with both readings). Where future bookings exist, add their estimated mileage to detect a breach *during* a booking. Always show the confidence tier — an estimate from platform median is not the same claim as one from two real readings on that car.

Excluded from any burn calculation: rentals with a swap (the delta spans vehicles), readings older than 180 days, and any vehicle whose readings regress.

7 • Data model

Four new tables, all namespaced `vehicle_*`, all `tenant_id NOT NULL` with the canonical four-policy RLS template `((tenant_id = get_user_tenant_id()) OR is_super_admin())` — even though neighbouring tables have RLS off, new

tables should be correct.

Table	Purpose & key columns	Notes
vehicle_odometer_readings	The observation stream. vehicle_id, reading, observed_at, source (handover / manual / customer / telematics / service), source_ref, recorded_by, unit, is_suspect	Carries vehicle_id explicitly — solves \$5.5. unit stamped at write time so a later miles→km switch cannot reinterpret history.
vehicle_maintenance_rules	What is due. vehicle_id (nullable = tenant default), rule_type, service_type, interval_miles, interval_months, lead_miles, lead_days, is_active	service_type must reuse the existing 14 strings from AddServiceRecordDialog so completed work matches back to the rule.
vehicle_maintenance_jobs	A piece of work. status (reported → scheduled → in_progress → awaiting_parts → completed → cancelled), priority, category, vendor_id, blocked_date_id, service_record_id, rental_id, acknowledged_by/at	Completion writes the service_records row and stores the FK. acknowledged_by replaces an assignment system — at 1–4 staff, visibility of who touched it last is enough.
vehicle_health_cache	Derived status per vehicle. status, reasons (jsonb), next_due_date, next_due_miles, confidence, computed_at	Denormalised so the fleet list is one query. Rebuilt by the triggers in \$6.2.

Changes to existing objects

Object	Change
check_rental_overlap()	Add blocked_dates clause with a distinct errcode, respecting check_start
sync_vehicle_maintenance_status()	Tenant-aware, timezone-aware, is_disposed guard, open-job guard, per-row via the transition RPC
update_vehicle_last_service()	COALESCE to newest non-NULL mileage so a mileage-less entry cannot destroy the baseline
rental_key_handovers	Add vehicle_id, populate handed_by
blocked_dates	Add source_type + reason_code (structured, safe to broadcast). Exclusion constraint scoped to maintenance-sourced rows. Keep narrative off this table.
vehicle_events	Backfill tenant_id (84 NULL rows); add a service_records trigger emitting the three existing unused enum values
reminders	New rule codes: SERVICE_DUE_*, SERVICE_OVERDUE, COMPLIANCE_*. object_type='Vehicle' exactly — live data has inconsistent casing and the dedupe index is case-sensitive
tenants	fleet_health_enabled, plus a jurisdiction field (\$12 D2). New booking-read columns need GRANT SELECT TO anon or every booking site 403s and falls back to default branding
vendors (P2)	Reusable garage entity so "Kwik Fit" is picked, not retyped

ALERT IDEMPOTENCY

The existing unique index is (rule_code, object_type, object_id, due_on, remind_on). Because remind_on is in the tuple, any generator anchoring it to today creates a fresh row nightly — which is what the MOT/TAX generators already do. For a **mileage** rule there is no meaningful date at all, so the index gives no protection. Key mileage alerts on the interval **ordinal** — floor((current - baseline) ÷ interval) — and resolve against the **maximum ordinal ever reached**, so a downward odometer correction cannot re-fire a closed alert.

8 · The end-to-end flows

F1 · Establish the baseline GATING REQUIREMENT

Nothing works without this, and it is the flow most likely to be skipped.

First run, per tenant

- 1 Operator enables Fleet Health in Settings (per-tenant flag, default off).
- 2 System computes coverage. If below threshold it renders a **setup screen**, never an all-unknown table.
- 3 Setup walks the vehicle list **ordered by urgency** — highest utilisation, oldest last-service, soonest compliance date — asking only for: odometer today, last service date, last service mileage. Three fields.
- 4 Progress is explicit and finite: "7 of 22 vehicles set up".
- 5 Compliance dates already present (~38 vehicles) light up immediately with zero input — the tenant sees value before finishing setup.
- 6 CSV import offered for tenants coming off a spreadsheet.
- 7 Pre-existing overdue items seed as **acknowledged/historical** at reduced severity with a distinct rule code — see the backfill warning below.

BACKFILL BLAST RADIUS

Across real vehicles, 25 already have a past registration date and 19 a past tax date. Switching generation on emits **~44 immediate criticals across a 173-vehicle fleet** — a quarter of every real car alarming at once — then near silence. Combined with `send-reminder-notifications` having **no tenant filter on either its reminders query or its app_users query** (a verified cross-tenant fan-out), an unguarded first run is an incident, not a launch. Require per-tenant opt-in before first generation, cap alerts per tenant per run, and fix the tenant scoping *before* routing anything new through it.

F2 · Configure a schedule

Opt-in, never a prerequisite

- 1 Tenant-level defaults may be set once (oil 5,000 mi / 6 mo, tyres 7,500 mi, brakes 10,000 mi, etc.).
- 2 Any vehicle may override or exclude a rule.
- 3 Rules are **strictly optional**. With zero configuration the product still surfaces compliance dates and last-service age — 12 of 23 tenants have ≤ 4 vehicles and will never configure intervals.
- 4 Mileage rules on a vehicle with no anchor render as **Unknown** with a one-tap "add a reading" action — not as "not due".

F3 · Odometer intake

Four sources, one stream

- 1 **Handover** (primary today). Fix the `if (mileage) zero` bug, add `vehicle_id`, populate `handed_by`, add a plausibility check against the last reading with an explicit override, and prefill from the vehicle's current value.
- 2 **Manual entry** **NEW, GATING** — an audited control on the vehicle page. This is the **only** way to seed a fleet or correct a bad value, and it does not exist today.
- 3 **Customer-reported** **DECISION D4** — a link in existing rental reminders to a simple authenticated page. Marked `source='customer'`, unverified. This is the only viable source for a 90-day rental or a lockbox tenant.
- 4 **Telematics** — deferred; the Tesla integration is dead in production and never read odometer anyway.

Write rule: the vehicle cache updates via `GREATEST(COALESCE(current_mileage,0), new_reading)` in a DB trigger with a tenant predicate. Corrections go through an explicit audited action that records the prior value and a reason.

F4 · Evaluation & forecast

Nightly sweep + trigger recompute

- 1 Resolve each active rule's anchor: last matching `service_records` row **by** `service_type` — never `vehicles.last_service_date`, which is "last any service" and cannot drive per-item intervals.
- 2 Compute remaining miles and remaining days.
- 3 Convert miles remaining into a projected date using §6.4, tagging the confidence tier.
- 4 Overlay confirmed future bookings and their estimated mileage; flag a projected breach *during* a specific booking.
- 5 Write the derived status and reasons to `vehicle_health_cache`.
- 6 Raise or resolve `reminders` rows against the ordinal key from §7.

F5 · Alerting

ALERT FATIGUE IS THE MEASURED CURRENT STATE, NOT A FUTURE RISK

99.1% of 14,448 notifications are unread. Snooze has **never been used once** across 598 reminders. Only 6 of 39 tenants have operator email enabled and only 4 have a recipient address — so the email path can reach **at most 4 operators**. There is no staff phone number column on `app_users` at all, and all SMS/WhatsApp plumbing is wired exclusively to customers. **There is currently no channel that can reach a human about a maintenance event.**

Recommendation: ship Fleet Health alerting as **pull-first** and say so plainly. A persistent count on the Vehicles nav item and amber/red rows in lists the operator already opens will outperform pushing into a 14,000-deep unread pile. Set an explicit per-tenant weekly alert budget and define what happens at the cap. If push is wanted later, it needs new infrastructure (a per-user contact number, a staff notification transport) — that is a scoped project, not a checkbox.

F6 · Taking a vehicle off the road THE CRITICAL FLOW

Scheduling a hold

- 1 Operator picks a window. The system proposes the best gap from the existing calendar data — Brandon's "Suggested slot: Tuesday, no bookings".
- 2 **Conflict check runs against ['Active', 'Pending']**. Today `blocked-dates-manager.tsx` checks `['Active', 'Confirmed']` — and **"Confirmed" does not exist in live data** while **"Pending" (25 live rentals) is never checked**. That is a real bug on the current screen.
- 3 Conflicts are shown **with the amount already paid**, and each must be resolved before the hold commits: swap, reschedule, or refund.
- 4 The hold commits inside a single `SECURITY DEFINER` RPC modelled on `swap_rental_vehicle` — job row, `blocked_dates` row and status transition in **one transaction**.
- 5 The block carries a structured `reason_code` only. Narrative stays on the tenant-scoped job table (`blocked_dates` is broadcast with RLS off).
- 6 Release happens when the job completes, not merely when the calendar window ends — and only if no *other* block still covers the vehicle.

TAKING AN ACTIVELY-RENTED VEHICLE OFF THE ROAD

There is **no mechanism** for this on a fixed-term rental. `payg_paused` exists and is respected, but only 1 of 73 open rentals is PAYG. The other 72 have no pause, no pro-rata credit and no substitution clause in the agreement — the three refund functions are all cancellation-shaped. **Recommendation for v1:** refuse to take an actively-rented vehicle off the road and force the `swap_rental_vehicle` flow instead, which at least reassigns the customer to a car. A PAYG hold should set `payg_paused` — the agreement already promises this and nothing implements it.

F7 · Doing the work

Deliberately thin in v1

- 1 A job carries priority, category, status, and free-text notes.
- 2 The garage is picked from a reusable vendor list (P2) rather than retyped — the current free-text `vendor` field is why the data is unusable.
- 3 "At the garage — expected back <date>" is a valid state that **does not require knowing what work is being done**. This is the realistic small-operator case.
- 4 An invoice is captured as a **photo of paper** into a private bucket. No parts/labour split in v1.
- 5 Anyone opening or acting on a job stamps `acknowledged_by`, visible to everyone else — cheap prevention for the two-staff-both-book-the-garage case.

F8 · Completion & settlement

The one path that must not be duplicated

- 1 Operator marks the job complete, entering date (defaulted to today), odometer (prefilled, one tap) and cost.
- 2 The system writes **one** `service_records` row and stores its id on the job.
- 3 Existing triggers then fire automatically: P&L posts idempotently as `service:<id>`, and the vehicle's last-service fields re-derive. **No new cost-posting code is written.**
- 4 The odometer entered also lands as an observation, resetting the mileage baseline for that rule.
- 5 The job's own block is released; if another block still covers the vehicle, it stays off road and the UI says **which** one is holding it.
- 6 The matching rule's next due is recomputed and its reminder resolved against the max ordinal.

DOUBLE-COUNT GUARD — REQUIRED

If the operator also logs the garage invoice as a `vehicle_expenses` row categorised Service, the cost **doubles** across every P&L surface, invisibly. Required: a nullable FK from `vehicle_expenses` to the job, and `handle_vehicle_expense_pnl` must skip its P&L insert when that FK is set. Ship a same-vehicle / ± 7 -day / similar-amount duplicate warning — it returns zero rows today, so it stays cheap.

Also: `upsert_service_pnl_entry` hardcodes `category='Service'` and ignores `expense_categories` entirely, while the expense trigger respects `pnl_bucket`. One tenant maps Service to a different bucket, so their maintenance spend splits across two P&L categories depending on which table it was entered in — and it will never reproduce in dev.

F9 · Return inspection → defect

Activating what is already built

- 1 At receiving handover, staff complete a short checklist alongside the odometer: tyres, warning lights, brakes, glass, body, interior, cleanliness.
- 2 Photos capture with `capture="environment"` so the camera opens directly, downscaled client-side before upload, with a **visible per-photo state** (queued / uploading / failed / retry). Today an oversized iPhone photo fails silently.
- 3 Outcome: Pass · Attention · Off road.
- 4 "Attention" or worse creates a `vehicle_maintenance_jobs` row linked to the rental — Brandon's "Issue #2041 reported after Rental #3842".
- 5 A critical defect proposes an immediate hold, which runs F6's conflict check.
- 6 **The gate must be deferrable.** A car returned at 19:00 on a Saturday cannot block deposit release until Monday — the Stripe authorisation has an expiry clock running. Default to "flag now in two taps, detail later" with a visible "awaiting inspection" state bounded by `deposit_hold_expires_at`.

LOCKBOX TENANTS

9 tenants have lockbox handover enabled, where **nobody is present** at either end. A handover-only inspection design degrades to **zero coverage** for them, and the product's own direction is toward more self-service. Their only viable paths are a customer-facing prompt (F3 step 3) or a periodic staff "yard walk" capture independent of any rental. This must be stated explicitly rather than assumed away.

F10 · Damage → renter recharge

Only after \$5.8 is fixed. Preconditions, all of them: bucket made private with signed URLs; `handed_by` / `uploaded_by` populated; damage reports made append-only with a version once a charge references them; and the evidence bundle

frozen at charge time — resolved URLs, content hashes, finding text, model, timestamp and named actor snapshotted onto the charge.

Money then flows through the existing paths (`apply-fine` or `capture-deposit-hold`). Note that neither applies sales tax, and 16 tenants have tax enabled at 3–13% — §12 D5.

F11 · Where the signal actually appears **HIGHEST LEVERAGE, LOWEST COST**

A solo operator will not open a Fleet Health tab. They open **Rentals** → **New**, pick a car, and hand over keys. If "inspection expired 4 days ago" is only visible on a tab they never open, the feature changes no behaviour even with perfect data.

Inline chips wherever a vehicle is chosen

- 1 The `rentals/new` vehicle picker — the single highest-value placement.
- 2 The swap-candidate list — which **already renders a "Blocked for maintenance" badge**, so the pattern exists to copy.
- 3 The vehicles list row and the fleet calendar.
- 4 Each placement declares whether it **warns** or **blocks**. A warning an operator can override with a reason will be adopted; a hard block on a car they need to rent today will be worked around or will kill the feature.

F12 · Reporting

Ship **cost**: maintenance spend per vehicle, per category, over time; cost per mile where mileage exists; days off road. Withhold **ratios** — maintenance-to-revenue and any disposal recommendation — until §5.6 is resolved. Also fix `get-vehicle-profitability`, which silently treats "all time" as the last 365 days, and which cannot distinguish "in the shop" from "idle" in its utilisation denominator.

F13 · Trax AI

Cheaper than expected — one SQL function

- 1 Extend `get_rag_metrics(p_tenant_id)` with maintenance aggregates. This is the channel Trax actually answers counting questions from, and it is injected into the prompt as authoritative.
- 2 Do **not** rely on the vector store: 80,565 sync rows are unprocessed, `rag-sync` has no cron and no caller, and the corpus is a frozen five-month-old snapshot covering 58 of 452 vehicles.
- 3 Optionally add a `schedule_maintenance` action to the existing propose → confirm → execute registry (which currently holds exactly one action).
- 4 Insight cards ("this vehicle is 186% above your fleet average") follow the `generate-review-summary` pattern — but must be phrased as cost observations, not disposal advice, until §5.6 is fixed.

9 · Edge-case register

Scenario	Why it breaks	Handling
Odometer legitimately 0 on a new vehicle	<code>if (mileage)</code> treats 0 as falsy — skips both the vehicle write and excess-mileage calc, silently	Guard on <code>!== null && !== undefined</code>
Reading goes backwards after a typo correction	Interval ordinal decreases; a resolved alert re-fires or is suppressed forever	<code>GREATEST</code> on the cache; resolve against max ordinal ever reached; audited correction action
Rental swapped mid-term (one live rental swapped twice)	Handovers have no <code>vehicle_id</code> and are <code>UNIQUE</code> on (rental, type) — the delta spans up to 3 cars	Add <code>vehicle_id</code> ; require a reading at swap; exclude swapped rentals from burn estimates

Scenario	Why it breaks	Handling
90-day rental with no reading	~15,000 unobserved miles (p90 ~29,000). 18 open rentals are past 90 days, one at 228	Customer-reported odometer prompt, or accept date-based only for long-term
Quick entry with no mileage	<code>update_vehicle_last_service</code> NULLs <code>last_service_mileage</code> — permanently destroys the baseline	<code>COALESCE</code> to newest non-NULL. 21 of 41 live records already have no mileage
Second overlapping block survives job completion	59 overlapping pairs live, no constraint; reconciler still matches; car never returns to sale	Release only the job's own block; show the union with sources; exclusion constraint on maintenance-sourced blocks
Rental returns at 01:00 UTC during the cron sweep	Bulk UPDATE races three unconditional close paths; last-writer-wins, undiagnosable	Single transition RPC + status audit row; per-vehicle, gated on job state
Hold ends today in Los Angeles	UTC <code>CURRENT_DATE</code> has already rolled; vehicle released 3–7h early on every hold	Evaluate in tenant timezone, or store an explicit end timestamp _{tz}
Vehicle disposed with an open job	<code>undo_vehicle_disposal</code> writes Available unconditionally from SQL — invisible to a TypeScript grep	Disposal cancels jobs and resolves reminders; undo does not resurrect them; use <code>COALESCE(is_disposed,false)=false</code> everywhere
Maintenance scheduled over a paid booking	Conflict check uses a status that does not exist and misses Pending; no refund path is cancellation-shaped for this	Check <code>['Active','Pending']</code> , show amounts paid, force explicit resolution
Two staff both act on one alert	No assignment or claim anywhere; produces two service records and double-counted cost	<code>acknowledged_by/at</code> stamped and visible; uniqueness constraint on service records
Operator with 3 vehicles	12 of 23 tenants have ≤ 4 vehicles; setup overhead exceeds value and guarantees abandonment	Zero-configuration value on day one; rules strictly opt-in
Fleet at 0% odometer coverage	An all-unknown table reads as broken software	Setup screen below threshold; never render all-unknown
8 HEIC photos, 3 over 10MB	Client silently returns; staff sees 5 thumbnails and walks away; evidence gap found weeks later in a dispute	Per-file error toast, client-side downscale, explicit per-photo state
Car at location A, operator at location B	8 tenants run multiple locations (one has 8 sites for 4 cars); nothing records where the car is	Show location on every row and filter by it; surface "last seen at" as inferred, not authoritative
Lockbox handover	No human present to inspect or read the odometer, at either end	Customer prompt or yard-walk capture; state the coverage limit explicitly
Saturday 19:00 return, garage opens Monday	A hard inspection gate strands the operator with a deposit authorisation expiring	Deferrable by default; bounded by <code>deposit_hold_expires_at</code>
Tenant switches miles → km or USD → AED	Every stored mileage is unitless and every amount currency-less; history silently reinterprets	Stamp unit and currency at write time; treat tenant settings as display resolvers only
Free-text reason on a block	<code>blocked_dates</code> is broadcast with RLS off; the anon key ships in the booking bundle	Structured <code>reason_code</code> on the block; narrative on the tenant-scoped table
Manager reaches an unregistered route	Route protection fails open; two edge functions carry stale allowlists	Register in <code>ROUTE_TO_TAB</code> ; or inherit the existing <code>vehicles</code> tab key — see §12 D3

10 · Non-functional requirements

10.1 The interaction budget TREAT AS A REQUIREMENT

This is the most likely cause of abandonment and it is measurable, not speculative. To log one service today an operator must open the portal on a phone, load a **1,673-line** client page, scroll past twelve unrelated cards, open a dialog, and type a date, a type from 14 options, a mileage they do not have, and a cost. Recording an odometer means loading a **7,756-line** page. There is **no PWA** (no manifest, no service worker), **no camera capture attribute anywhere** in the portal, and no offline buffer. The evidence that this already fails: 41 service records across 8 tenants, 27 of them from a single tenant.

BUDGET

Logging a completed service or an odometer reading must be **≤3 taps from the portal home on a phone**, must not require loading either mega-page, and must work one-handed. That implies a global "quick log" entry point (vehicle picker → number pad → done). Cost the lightweight route as part of the feature, not as a follow-up.

10.2 Form friction is currently inverted

`service_type` is **required** with 14 fixed options while `mileage` is **optional** — exactly backwards. Mileage is the field with downstream consequences; type is the field operators cannot be bothered with. The live data proves it: 22 of 41 records are typed "Other"/NULL and 21 have no mileage. Default the date to today, make type optional with a free-text fallback, and push hardest on mileage (prefilled, one tap to confirm).

10.3 Data retention

1,119 condition photos already sit in a world-readable bucket with no TTL, and **zero cron jobs delete anything**. The privacy policy contains no reference to photos, images or retention. Adding maintenance photography multiplies an already-undisclosed indefinite retention of imagery containing plates, locations and bystanders — and silently defeats the existing `hide_vehicle_registration` feature. A retention period tied to the dispute window, the mechanism that enforces it, and the disclosing policy sentence are all in scope.

10.4 Never certify safety

The product must not output a binary "safe" state. Model it as "inspection recorded by <named user> at <time> against <checklist version>", with the checklist stored immutably and explicit portal copy that this is an **operational log, not a roadworthiness certification**. Require an actor id on every inspection record from day one — it is unbackfillable, as the 1,279 existing actor-less rows demonstrate. The rental agreement contains no roadworthiness warranty and no disclaimer, so a Fleet Health screen would be the closest thing to a safety representation this product has ever made.

10.5 Conventions to follow

- React Query keys include tenant: `["vehicle-health", tenant?.id, ...]`, with `enabled: !!tenant`.
- Follow `use-expenses.ts` for the CRUD + storage + `DEPENDENT_KEYS` invalidation fan-out pattern; `vehicle-owners` for page anatomy; `reminders` for stats + filters + bulk actions.
- Schema changes via `mcp__supabase__apply_migration`, never hand-written files — the repo has 69 files with no live row and 43 live rows with no file, so `db push` would replay DDL that already exists.
- Regenerate types and copy to **all four** apps (portal, booking, admin, bonzah).
- Tests are pure-logic Vitest only — nothing renders, because portal pins React 18 while the root hoists React 19.
- Design tokens: indigo `#6366f1` accent, `#eef2ff` table headers, 1px `#f1f5f9` borders, no shadows, DM Sans.

11 · Phasing

Phase	Contents	Why this order	Exit criteria
P0 Fix	The \$5 blockers: <code>blocked_dates</code> in <code>check_rental_overlap</code> ; timezone + tenant + disposal guards on the reconciler; single transition RPC + status audit; odometer zero/monotonic/tenant fixes; <code>vehicle_id</code> on handovers; <code>COALESCE</code> on last-service; tenant scoping on <code>send-reminder-notifications</code> ; conflict check status values	Every one of these is a correctness bug that exists today , independent of Fleet Health. Several are actively armed. Building on top of them makes them harder to fix and makes Fleet Health look like the cause.	A hold cannot be bypassed by any booking path, including the portal form
P1 Ship	Manual odometer entry + bulk setup + CSV import; compliance surface (mount the three orphaned components); health status + reasons + Unknown ; date-based rules; inline chips at point of rental; hold flow with conflict resolution; completion → <code>service_records</code> ; maintenance on the existing calendar; cost reporting	Delivers value with zero configuration using data 23 tenants already have, and unblocks the mileage work by making capture possible. Roughly a third of this is mounting components that already exist.	A tenant with no setup sees something true and useful on first open
P2 Extend	Mileage-interval rules + forecasting; booking-conflict projection; return inspection checklist + photo hardening; jobs with vendors; Trax metrics; quick-log mobile route	Depends on P1 capture existing and on odometer coverage having actually improved. Gate the mileage rules on a measured coverage threshold, not on a date.	Coverage above threshold on real tenants; forecast within tolerance
P3 Elaborate	Work orders with parts/labour; damage → claim with a frozen evidence bundle; accounting sync for maintenance; profitability ratios; telematics	Each carries an unresolved dependency — ledger correctness, evidence hardening, a payables path in the accounting sync, or a dead integration.	Blocked until \$5.6 and \$5.8 are resolved

12 · Decisions required from you

These are product and liability calls, not engineering ones. Each one changes what gets built.

#	Decision	Context	Recommendation
D1	Does expired legal compliance hard-block a rental, or warn?	Renting an uninspected/unregistered vehicle is strict liability for the operator and typically voids their motor policy. Nothing enforces it today. But a hard block on a car they need to rent today is the classic reason a feature gets worked around.	Hard block, server-side, with a reasoned override that is audited. This is the strongest adoption argument and the highest liability gap.
D2	Which jurisdictions does v1 support?	Fields are UK (MOT); fleet is US. No country/state on <code>tenants</code> ; <code>garaging_state</code> is demo-only. State inspection cadences differ fundamentally, and Florida has none.	Model generically (<code>obligation_type</code> , <code>jurisdiction</code> , <code>due_date</code> , <code>cadence</code>) and ship with operator-entered dates rather than inferred rules. Do not hardcode MOT.
D3	New <code>maintenance</code> permission tab, or inherit <code>vehicles</code> ?	4 manager users and 2 ops users exist platform-wide against 47 <code>head_admins</code> . A new tab key means <code>permissions.ts</code> changes, two stale edge-function allowlists, and a backfill — serving 6 people.	Inherit <code>vehicles</code> . Zero permissions work, zero redeploy, zero backfill. Revisit only when a tenant asks.

#	Decision	Context	Recommendation
D4	Is a customer-facing odometer prompt in scope?	The renter is the only person who reliably knows the mileage on a 90-day rental, and the only possible source for a lockbox tenant. No table, route, or UI exists today.	Yes, in P2. Without it, mileage rules are unusable for long-term and lockbox — a growing share of the business.
D5	Are damage/maintenance recharges taxable?	16 tenants collect 3–13% sales tax on rentals. <code>apply-fine</code> and the deposit capture apply none, and no ledger has a tax column.	State explicitly on the charge UI that recharges are tax-exclusive and handled outside the product — or scope the tax line properly. Do not leave it implicit; operators will assume it matches rentals.
D6	Is the tax-as-Revenue classification known and accepted?	\$203k of collected tax and \$1k of deposits sit on the Revenue side of <code>pnl_entries</code> , inflating every per-vehicle profit figure.	Confirm before any profitability surface ships. If accepted, Fleet Health ships cost only and makes no ratio claims.
D7	Pull-only alerting, or build a staff notification transport?	No staff phone column exists; email reaches at most 4 operators; 99.1% of in-app notifications are unread.	Pull-only for v1 , stated plainly. Treat push as a separate scoped project rather than an assumed capability.
D8	Can maintenance take an actively-rented vehicle off the road?	No pause, pro-rata credit or substitution clause exists for the 72 fixed-term open rentals. Refund functions are cancellation-shaped.	No in v1 — force the swap flow. Add a substitution/downtime clause to the agreement template before revisiting.

ONE CLOSING NOTE ON SEQUENCING

The P0 fixes are worth doing whether or not Fleet Health is ever built. A maintenance hold that any booking path can bypass, holds that release hours early for every tenant, odometer readings that silently vanish when the value is zero, and four live rentals armed to produce corrupt mileage on their next return — these are defects in the product as it stands today. Fleet Health is the reason to finally fix them, which is a good reason to start there.